

APP-5258 · Enrolamiento biométrico Facephi para Soft Token

Microservicio: `onboarding-micro-person`

Stack: .NET 8 (ASP.NET Core) + MongoDB.Driver

Épica: APP-5222 · **Bloquea:** APP-5223 — Enrolamiento de Soft Token mediante validación biométrica Facephi

Documento de planeación técnica — 10 de agosto de 2026

Cómo usar este documento. Está pensado para leerse antes de tocar el repositorio y luego ejecutarse de arriba a abajo. Las secciones 1 a 6 son diseño y decisiones; la sección 7 contiene el código C# completo, archivo por archivo, listo para adaptar; las secciones 8 a 12 cubren configuración, seguridad, pruebas, subtareas y las preguntas que hay que cerrar con el equipo.

Advertencia importante: el código de este documento **no ha sido compilado**. Se escribió siguiendo las convenciones estándar de ASP.NET Core 8 y `MongoDB.Driver`, pero sin acceso al repositorio real ni al SDK de .NET. Espera ajustar *namespaces*, *usings* y el registro en `Program.cs` a lo que ya exista en el micro.

Índice

1. Contexto y alcance
 2. Arquitectura y componentes
 3. Modelo de datos
 4. Contratos de API
 5. Flujo backend y máquina de estados
 6. Checklist de descubrimiento en el repositorio
 7. Código C# completo
 8. Configuración e integración (`Program.cs` , `appsettings.json`)
 9. Seguridad y datos biométricos
 10. Pruebas y verificación
 11. Subtareas y estimación
 12. Preguntas abiertas
-

1. Contexto y alcance

APP-5258 define el backend que gestiona el **enrolamiento biométrico previo a la activación del Soft Token**. El flujo, tal como lo describe el ticket:

1. Se crea el registro del enrolamiento en MongoDB.
2. Se almacena el token de la imagen frontal del documento.
3. Se almacena el token de la imagen trasera del documento.
4. Una vez capturada la biometría facial, se invoca la validación con Facephi utilizando la información almacenada.
5. Si la validación es exitosa, se actualiza la información de *tracking* y el flujo continúa con el enrolamiento del Soft Token.
6. Posteriormente, el contrato firmado y las evidencias biométricas serán almacenados en Alfresco.

1.1 Punto de partida real

Facephi **ya está integrado y en producción** dentro de `onboarding-micro-person` para el proceso de onboarding. APP-5258 **no levanta un microservicio nuevo**: son ajustes dentro del mismo micro para que esa integración sirva también al flujo de Soft Token.

De ahí la regla que gobierna todo el trabajo:

Reutilizar lo que ya existe (cliente Facephi, credenciales, manejo de errores, autenticación, logging) y **crear únicamente lo específico de Soft Token**.

La única pieza claramente nueva a nivel de infraestructura es la persistencia: según los requisitos se crea una **base de datos MongoDB nueva**, aislada de la que usa onboarding.

1.2 Dentro y fuera de alcance

Dentro de APP-5258	Fuera (fases posteriores)
Registro de enrolamiento en la BD Mongo nueva	Activación del Soft Token (APP-5223)
Guardado de tokens de documento (frontal y trasero)	Almacenamiento del contrato firmado en Alfresco
Invocación de la validación biométrica en Facephi	Almacenamiento de evidencias biométricas en Alfresco
Persistencia del <code>tracking</code> devuelto por Facephi	Firma del contrato
Consulta del estado del proceso por <code>idT24</code>	Frontend / app móvil

`FacephiClient` y `AlfrescoClient` están marcados en el ticket como "implementación posterior". En este plan:

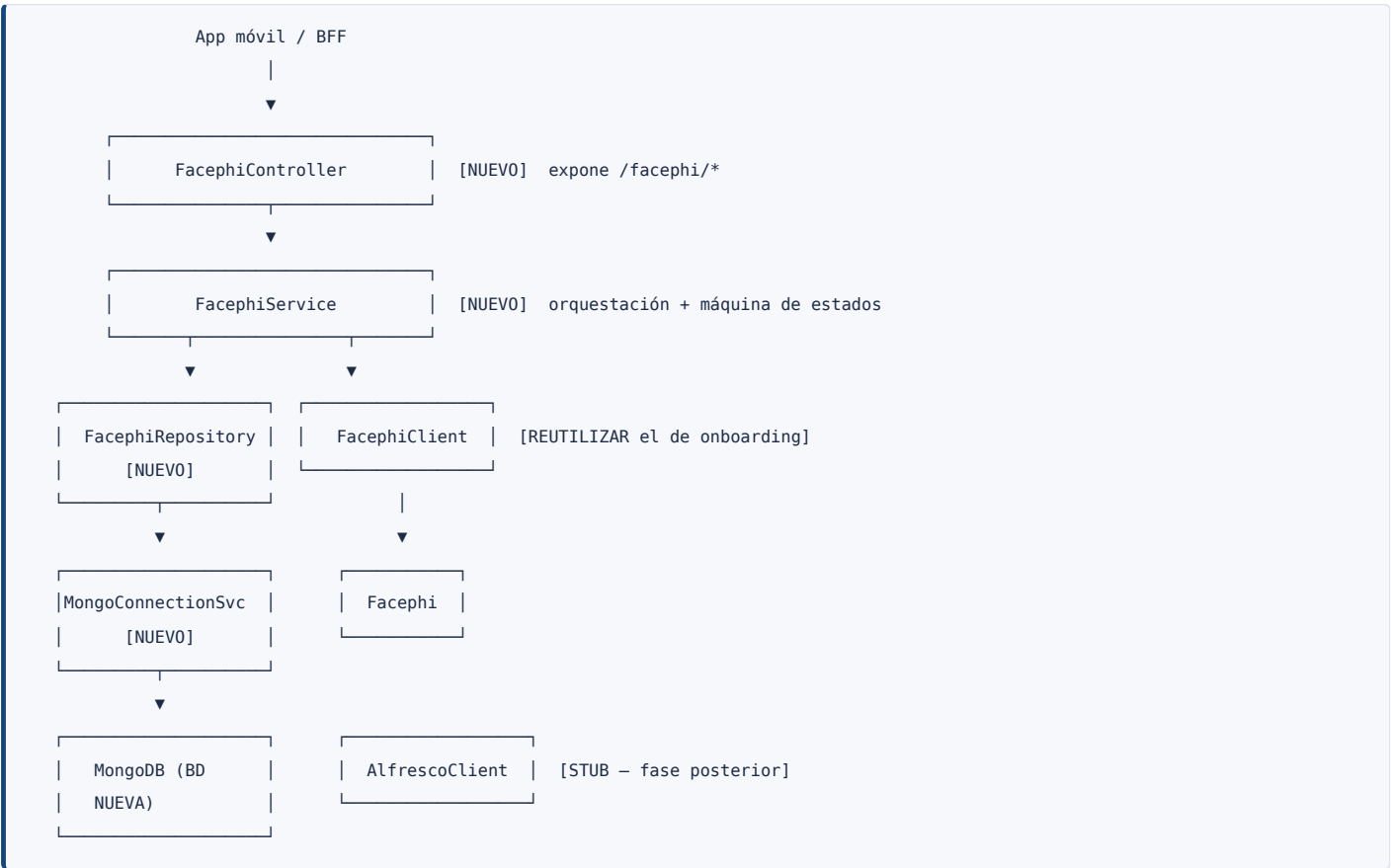
- `AlfrescoClient` se entrega como interfaz + *stub* documentado — nada más, es fase posterior.
- `FacephiClient` es distinto: como Facephi ya está integrado en el micro, lo correcto **no** es crear un cliente nuevo, sino **extender el existente**. El documento incluye un esqueleto por si resulta que el cliente actual no es reutilizable, pero la primera opción siempre es reutilizar.

1.3 Criterios de aceptación propuestos

El ticket viene con *Acceptance Criteria*: *None*. Propuesta para completarlo:

- Dado un `idT24` válido, `POST /facephi/enrollment` crea el registro en la BD nueva y responde `201`.
- Guardar el documento frontal y luego el trasero persiste `token1` y `token2` en el mismo registro.
- Intentar guardar el documento trasero sin haber guardado el frontal responde `409`.
- Con ambos documentos guardados, `POST /facephi/validate` invoca Facephi y, si la validación es exitosa, persiste `tracking.operationId` y `tracking.extraData`.
- Si la validación falla, el registro queda marcado como fallido y el endpoint responde `422`.
- `GET /facephi/enrollment/{idT24}` devuelve el estado del proceso **sin exponer** los tokens biométricos.
- Ningún log del microservicio contiene `token1`, `token2`, el token facial ni `extraData`.

2. Arquitectura y componentes



2.1 Responsabilidades

Componente	Responsabilidad	Estado
FacephiController	Expone los endpoints del flujo. Sin lógica de negocio: valida el request, delega y traduce a códigos HTTP	Nuevo
FacephiService	Orquesta el proceso de enrolamiento biométrico. Dueño de la máquina de estados y de las reglas	Nuevo
FacephiRepository	Almacena y consulta la información del enrolamiento	Nuevo
MongoConnectionService	Establece la conexión con MongoDB y obtiene la colección correspondiente	Nuevo (apunta a la BD nueva)
FacephiClient	Integración con Facephi	Ya existe para onboarding → extender
AlfrescoClient	Contrato firmado y evidencias biométricas	Stub — fase posterior

2.2 Estructura de archivos

[N] nuevo · [?] probablemente ya existe en el micro → **fusionar, no duplicar**

```

Onboarding.Micro.Person/
├─ Controllers/
|   └─ FacephiController.cs [N]
├─ Services/
|   ├── IFacephiService.cs · FacephiService.cs [N]
|   ├── IMongoConnectionService.cs [N]
|   ├── MongoConnectionService.cs [N]
|   └─ MongoIndexInitializer.cs [N]
├─ Repositories/
|   └─ IFacephiRepository.cs · FacephiRepository.cs [N]
├─ Clients/
|   ├── IFacephiClient.cs · FacephiClient.cs [?] reutilizar el existente
|   └─ IAlfrescoClient.cs · AlfrescoClient.cs [N] stub
├─ Models/
|   ├── FacephiEnrollment.cs · TrackingInfo.cs [N]
|   └─ EnrollmentStatus.cs · DocumentType.cs [N]
├─ Dtos/
|   ├── CreateEnrollmentRequest.cs · CreateEnrollmentResponse.cs [N]
|   ├── SaveDocumentRequest.cs · SaveDocumentResponse.cs [N]
|   ├── ValidateRequest.cs · ValidateResponse.cs [N]
|   └─ EnrollmentStatusResponse.cs [N]
├─ Configuration/
|   ├── FacephiMongoSettings.cs [N]
|   └─ FacephiSettings.cs [?]
├─ Exceptions/
|   ├── EnrollmentNotFoundException.cs [N]
|   ├── InvalidEnrollmentStateException.cs [N]
|   └─ FacephiIntegrationException.cs [N]
├─ Middleware/
|   └─ ExceptionHandlingMiddleware.cs [?]
├─ Program.cs [?] solo añadir líneas
└─ appsettings.json [?] solo añadir sección

```

3. Modelo de datos

Base de datos MongoDB nueva · Colección: facephi_enrollments

```
{
  "idT24": "123456789",
  "documentType": "CED",
  "token1": "BAMBAQLNHJowGPjfeuDI...",
  "token2": "BAMBAQLNHJowGPjfeuDI...",
  "method": 1,
  "tracking": {
    "extraData": "BQABAQQ2gBNjuHN4...",
    "operationId": "123e4567-e89b-12d3-a456-426614174000"
  },
  "status": "BACK_SAVED",
  "createdAt": "2026-08-10T14:30:00Z",
  "updatedAt": "2026-08-10T14:32:10Z"
}
```

Campo	Tipo	Descripción
idT24	string	Identificador del cliente en T24. Clave de negocio , índice único
documentType	string	CED PASSPORT
token1	string	Token de la imagen frontal del documento
token2	string	Token de la imagen trasera del documento
method	int	Método de captura
tracking.extraData	string	Información devuelta por Facephi durante la validación
tracking.operationId	string (UUID)	Identificador de la operación en Facephi
status	string	Estado del proceso — recomendación, ver 3.1
createdAt	date	Fecha de creación — recomendación
updatedAt	date	Última modificación — recomendación

3.1 [recomendación](#): campos status , createdAt y updatedAt

Estos tres campos NO están en el modelo del ticket. Son una propuesta de este documento y requieren el visto bueno del equipo antes de implementarse.

Motivo. El ticket pide un endpoint GET /facephi/enrollment/{idT24} que "consulta el estado del proceso", pero el modelo no tiene ningún campo de estado. Sin él, el estado hay que inferirlo de qué campos están presentes, lo cual funciona pero tiene tres inconvenientes: no distingue "validación fallida" de "aún no validado", no deja rastro de cuándo ocurrió cada paso, y obliga a repetir la lógica de inferencia en cada consumidor. createdAt además es necesario para el índice TTL de retención (sección 9).

Alternativa si el equipo prefiere ceñirse al modelo del ticket. El estado se deriva así:

Condición	Estado reportado
tracking.operationId presente	VALIDATED
token1 y token2 presentes	BACK_SAVED
solo token1 presente	FRONT_SAVED
ninguno presente	CREATED

Esa inferencia se implementa en un solo método de `FacephiService` y el resto del código no cambia. Se pierde el estado `VALIDATION_FAILED` (no habría forma de distinguirlo) y la retención por TTL hay que resolverla con otro mecanismo. El código de la sección 7 usa la versión `con status` ; migrar a la variante inferida es sustituir un método.

3.2 Índices

Índice	Tipo	Motivo
<code>{ idT24: 1 }</code>	Único	Un enrolamiento vivo por cliente; el GET consulta por este campo
<code>{ createdAt: 1 }</code>	TTL (<code>expireAfterSeconds</code> configurable, sugerido 24 h)	No retener PII biométrica de flujos abandonados
<code>{ status: 1, createdAt: -1 }</code>	Compuesto	Consultas de soporte y monitoreo ("cuántos enrolamientos fallaron hoy")

Cuidado con el TTL: borra el documento completo pasado el plazo, incluidos los enrolamientos ya validados. Si el registro debe sobrevivir a la activación del Soft Token, hay dos opciones: subir el TTL a un plazo que cubra el proceso completo, o limpiar `token1` / `token2` al validar y quitar el TTL. **Decisión pendiente con el equipo** (ver sección 12).

4. Contratos de API

Base: `/facephi` · Todos los errores se devuelven como `ProblemDetails` (RFC 7807).

#	Método	Endpoint	Descripción
1	POST	<code>/facephi/enrollment</code>	Crea el registro inicial del enrolamiento
2	POST	<code>/facephi/document/front</code>	Guarda el token de la imagen frontal (<i>tentativo</i>)
3	POST	<code>/facephi/document/back</code>	Guarda el token de la imagen trasera (<i>tentativo</i>)
4	POST	<code>/facephi/validate</code>	Ejecuta la validación biométrica con los documentos almacenados y la captura facial
5	GET	<code>/facephi/enrollment/{idT24}</code>	Consulta el estado del proceso

Los endpoints 2 y 3 están marcados como **tentativos** en el ticket. Se implementan separados, pero toda la lógica vive en `FacephiService`: unificarlos después en un solo `POST /facephi/document` con un campo `side` es un cambio de controlador, sin tocar negocio ni persistencia.

4.1 POST /facephi/enrollment

POST `/facephi/enrollment`
Content-Type: `application/json`

```
{
  "idT24": "123456789",
  "documentType": "CED",
  "method": 1
}
```

201 Created

```
{
  "idT24": "123456789",
  "status": "CREATED",
  "createdAt": "2026-08-10T14:30:00Z"
}
```

Error	Cuándo
400	<code>idT24</code> vacío, <code>documentType</code> fuera de <code>CED / PASSPORT</code> , <code>method</code> inválido
409	Ya existe un enrolamiento validado para ese <code>idT24</code>

Idempotencia: si existe un enrolamiento **no validado** para el mismo `idT24`, se reinicia (se sobrescribe). Esto evita registros huérfanos cuando el cliente abandona el flujo en la app y vuelve a empezar.

4.2 POST /facephi/document/front y /facephi/document/back

POST `/facephi/document/front`
Content-Type: `application/json`

```
{
  "idT24": "123456789",
  "token": "BAMBAQLNHJowGPjfeuDI..."
}
```

200 OK


```
{ "idT24": "123456789", "status": "FRONT_SAVED" }
```

Error	Cuándo
400	<code>token</code> vacío o excede el tamaño máximo configurado
404	No existe enrolamiento para ese <code>idT24</code>
409	Transición inválida (p. ej. guardar el dorso sin el frente, o el enrolamiento ya está validado)

4.3 POST /facephi/validate

POST /facephi/validate
Content-Type: application/json

```
{
  "idT24": "123456789",
  "facialToken": "BQABAQQ2gBNjuHN4..."
}
```

200 OK — validación exitosa

```
{
  "idT24": "123456789",
  "status": "VALIDATED",
  "operationId": "123e4567-e89b-12d3-a456-426614174000",
  "success": true
}
```

Error	Cuándo
400	<code>facialToken</code> vacío o excede el tamaño máximo
404	No existe enrolamiento para ese <code>idT24</code>
409	Faltan <code>token1</code> o <code>token2</code> — no se puede validar sin ambos documentos
422	Facephi respondió, pero la validación biométrica no fue exitosa
502	Facephi no disponible, timeout o respuesta inesperada

La distinción entre `422` y `502` es deliberada: `422` es "la persona no coincide" (el negocio decide qué hacer, reintentar o rechazar); `502` es "no pudimos preguntar" (reintentable, monitorizable).

4.4 GET /facephi/enrollment/{idT24}

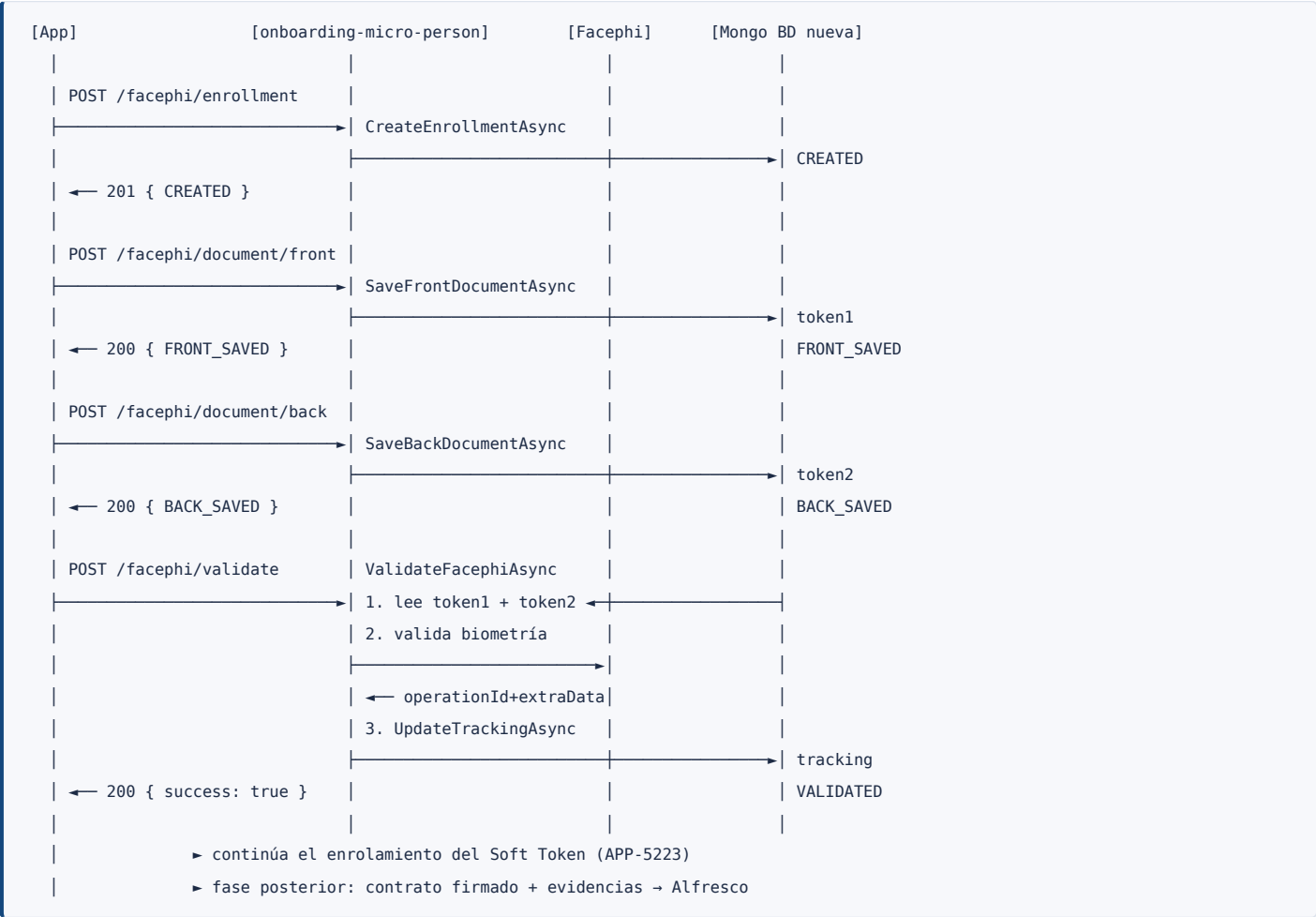
200 OK

```
{
  "idT24": "123456789",
  "documentType": "CED",
  "method": 1,
  "status": "VALIDATED",
  "hasFrontDocument": true,
  "hasBackDocument": true,
  "operationId": "123e4567-e89b-12d3-a456-426614174000",
  "createdAt": "2026-08-10T14:30:00Z",
  "updatedAt": "2026-08-10T14:35:42Z"
}
```

Este endpoint nunca devuelve token1 , token2 , el token facial ni extraData . Solo banderas booleanas y el operationId . Son datos biométricos: no deben salir del microservicio.

Error	Cuándo
404	No existe enrolamiento para ese idT24

5. Flujo backend y máquina de estados



5.1 Máquina de estados



Operación	Estados de origen permitidos	Estado resultante
CreateEnrollmentAsync	(no existe) · cualquiera excepto VALIDATED	CREATED
SaveFrontDocumentAsync	CREATED · FRONT_SAVED · BACK_SAVED · VALIDATION_FAILED	FRONT_SAVED
SaveBackDocumentAsync	FRONT_SAVED · BACK_SAVED · VALIDATION_FAILED	BACK_SAVED
ValidateFacephiAsync	BACK_SAVED · VALIDATION_FAILED	VALIDATED o VALIDATION_FAILED

Cualquier transición fuera de esta tabla → 409 Conflict . Volver a guardar el documento frontal cuando ya se guardó el trasero **retrocede** el estado a FRONT_SAVED : es intencional, obliga a recapturar el dorso y evita validar con una combinación inconsistente de imágenes.

6. Checklist de descubrimiento en el repositorio

Ejecutar esto antes de escribir la primera línea de código. El objetivo es no duplicar nada de lo que el micro ya tiene resuelto para onboarding.

#	Qué buscar	Comando	Si existe	Si no existe
1	Cliente Facephi actual	<code>grep -ril "facephi" --include=*.cs</code>	Reutilizar: añadirle la operación de validación que necesita Soft Token	Crear <code>IFacephiClient</code> (§7.12)
2	Configuración de Facephi	<code>grep -i "facephi" appsettings*.json</code>	Reusar sección, URL y credenciales	Crear <code>FacephiSettings</code> (§7.13)
3	Conexión a Mongo	<code>grep -rn "IMongoClient\ IMongoDatabase\ MongoClient" --include=*.cs</code>	Replicar el patrón registrado para apuntar a la BD nueva	Crear <code>MongoConnectionService</code> (§7.9)
4	Manejo de errores	<code>grep -rn "ProblemDetails\ ExceptionMiddleware\ ExceptionHandler" --include=*.cs</code>	Lanzar excepciones que ese middleware ya traduce	Añadir <code>ExceptionHandlingMiddleware</code> (§7.19)
5	Autenticación	<code>grep -rn "Authorize\ JwtBearer\ AddAuthentication" --include=*.cs</code>	Aplicar la misma política al <code>FacephiController</code>	Preguntar al equipo (§12)
6	Colisión de rutas	<code>grep -rn "[Route(" --include=*.cs</code>	Ajustar el prefijo si <code>/facephi</code> ya está tomado	—
7	Logging estructurado	<code>grep -rn "ILogger<" --include=*.cs \ head</code>	Seguir el mismo patrón y niveles	—
8	Pruebas existentes	<code>ls **/*Tests*/</code>	Añadir las pruebas nuevas al proyecto existente	Crear proyecto de pruebas
9	Versión de MongoDB.Driver	<code>grep -rn "MongoDB.Driver" *.csproj</code>	Usar la API de esa versión	Añadir el paquete

Reglas de fusión

- `Program.cs` y `appsettings.json` **nunca se reemplazan**: solo se añaden las líneas de la sección 8.
- Si el punto 1 encuentra un cliente Facephi utilizable, **descarta** `FacephiClient.cs` de este documento y usa `IFacephiClient` únicamente como la forma que `FacephiService` espera — o adapta directamente `FacephiService` a la interfaz existente.
- Si el punto 3 encuentra ya un `MongoConnectionService`, extiéndelo para que pueda entregar colecciones de la BD nueva en lugar de crear un segundo servicio con el mismo nombre.

7. Código C# completo

Namespace base asumido: `Onboarding.Micro.Person`. **Ajústalo al del micro real.** Paquetes NuGet necesarios (probablemente ya presentes): `MongoDB.Driver`, `Swashbuckle.AspNetCore`.

Orden de implementación sugerido: 7.1 → 7.14 (de las hojas hacia la raíz, cada archivo solo depende de los anteriores).

7.1 Configuration/FacephiMongoSettings.cs

```
namespace Onboarding.Micro.Person.Configuration;

/// <summary>
/// Configuración de la base de datos MongoDB NUEVA usada por el enrolamiento
/// biométrico de Soft Token. Es independiente de la BD que usa onboarding.
/// </summary>
public class FacephiMongoSettings
{
    public const string SectionName = "FacephiMongo";

    /// <summary>Cadena de conexión. NO se versiona: viene de variable de entorno o vault.</summary>
    public string ConnectionString { get; set; } = string.Empty;

    /// <summary>Nombre de la base de datos nueva.</summary>
    public string DatabaseName { get; set; } = string.Empty;

    /// <summary>Colección de enrolamientos.</summary>
    public string EnrollmentsCollectionName { get; set; } = "facephi_enrollments";

    /// <summary>Horas antes de que un enrolamiento expire por TTL. 0 = sin TTL.</summary>
    public int EnrollmentTtlHours { get; set; } = 24;

    /// <summary>Tamaño máximo permitido para un token de imagen (bytes del string base64).</summary>
    public int MaxTokenLengthBytes { get; set; } = 8 * 1024 * 1024;
}
```

7.2 Models/DocumentType.cs

```
using System.Text.Json.Serialization;

namespace Onboarding.Micro.Person.Models;

/// <summary>Tipo de documento de identidad usado en el enrolamiento.</summary>
[JsonConverter(typeof(JsonStringEnumConverter))]
public enum DocumentType
{
    CED = 1,
    PASSPORT = 2
}
```

7.3 Models/EnrollmentStatus.cs

```
using System.Text.Json.Serialization;

namespace Onboarding.Micro.Person.Models;

/// <summary>
/// Δ RECOMENDACIÓN – no forma parte del modelo original del ticket APP-5258.
/// Ver sección 3.1: pendiente de aprobación del equipo.
/// </summary>
[JsonConverter(typeof(JsonStringEnumConverter))]
public enum EnrollmentStatus
{
    CREATED,
    FRONT_SAVED,
    BACK_SAVED,
    VALIDATED,
    VALIDATION_FAILED,

    /// <summary>Fase posterior: contrato firmado y evidencias almacenados en Alfresco.</summary>
    CONTRACT_STORED
}
```

7.4 Models/TrackingInfo.cs

```
using MongoDB.Bson.Serialization.Attributes;

namespace Onboarding.Micro.Person.Models;

/// <summary>Información devuelta por Facephi durante la validación.</summary>
public class TrackingInfo
{
    [BsonElement("extraData")]
    [BsonIgnoreIfNull]
    public string? ExtraData { get; set; }

    [BsonElement("operationId")]
    [BsonIgnoreIfNull]
    public string? OperationId { get; set; }
}
```

7.5 Models/FacephiEnrollment.cs

```
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace Onboarding.Micro.Person.Models;

/// <summary>
/// Documento persistido en la colección "facephi_enrollments" de la BD nueva.
/// </summary>
public class FacephiEnrollment
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    [BsonIgnoreIfNull]
    public string? Id { get; set; }

    /// <summary>Identificador del cliente en T24. Clave de negocio (índice único).</summary>
    [BsonElement("idT24")]
    public string IdT24 { get; set; } = string.Empty;

    [BsonElement("documentType")]
    [BsonRepresentation(BsonType.String)]
    public DocumentType DocumentType { get; set; }

    /// <summary>Token de la imagen FRONTAL del documento.
    /// Dato biométrico: nunca se loguea ni se expone.</summary>
    [BsonElement("token1")]
    [BsonIgnoreIfNull]
    public string? Token1 { get; set; }

    /// <summary>Token de la imagen TRASERA del documento.
    /// Dato biométrico: nunca se loguea ni se expone.</summary>
    [BsonElement("token2")]
    [BsonIgnoreIfNull]
    public string? Token2 { get; set; }

    /// <summary>Método de captura.</summary>
    [BsonElement("method")]
    public int Method { get; set; }

    [BsonElement("tracking")]
    [BsonIgnoreIfNull]
    public TrackingInfo? Tracking { get; set; }

    // ————— △ RECOMENDACIÓN (sección 3.1) — pendiente de aprobación —————

    [BsonElement("status")]
    [BsonRepresentation(BsonType.String)]
    public EnrollmentStatus Status { get; set; } = EnrollmentStatus.CREATED;

    [BsonElement("createdAt")]
    public DateTime CreatedAt { get; set; }

    [BsonElement("updatedAt")]
    public DateTime UpdatedAt { get; set; }

    // —————
```

```
[BsonIgnore]
public bool HasFrontDocument => !string.IsNullOrEmpty(Token1);

[BsonIgnore]
public bool HasBackDocument => !string.IsNullOrEmpty(Token2);
}
```

7.6 Dtos/

Un archivo por DTO; se agrupan aquí para abreviar.


```

using System.ComponentModel.DataAnnotations;
using Onboarding.Micro.Person.Models;

namespace Onboarding.Micro.Person.Dtos;

// ----- POST /facephi/enrollment -----

public class CreateEnrollmentRequest
{
    [Required(ErrorMessage = "idT24 es obligatorio.")]
    [StringLength(50, MinimumLength = 1)]
    public string IdT24 { get; set; } = string.Empty;

    [Required(ErrorMessage = "documentType es obligatorio (CED | PASSPORT).")]
    [EnumDataType(typeof(DocumentType))]
    public DocumentType DocumentType { get; set; }

    [Range(0, int.MaxValue, ErrorMessage = "method debe ser un entero no negativo.")]
    public int Method { get; set; }
}

public class CreateEnrollmentResponse
{
    public string IdT24 { get; set; } = string.Empty;
    public EnrollmentStatus Status { get; set; }
    public DateTime CreatedAt { get; set; }
}

// ----- POST /facephi/document/front | /back -----

public class SaveDocumentRequest
{
    [Required(ErrorMessage = "idT24 es obligatorio.")]
    public string IdT24 { get; set; } = string.Empty;

    /// <summary>Token de la imagen del documento devuelto por el SDK de Facephi.</summary>
    [Required(ErrorMessage = "token es obligatorio.")]
    public string Token { get; set; } = string.Empty;
}

public class SaveDocumentResponse
{
    public string IdT24 { get; set; } = string.Empty;
    public EnrollmentStatus Status { get; set; }
}

// ----- POST /facephi/validate -----

public class ValidateRequest
{
    [Required(ErrorMessage = "idT24 es obligatorio.")]
    public string IdT24 { get; set; } = string.Empty;

    /// <summary>Token de la captura facial (selfie) devuelto por el SDK de Facephi.</summary>
    [Required(ErrorMessage = "facialToken es obligatorio.")]
    public string FacialToken { get; set; } = string.Empty;
}

```

```

public class ValidateResponse
{
    public string IdT24 { get; set; } = string.Empty;
    public EnrollmentStatus Status { get; set; }
    public string? OperationId { get; set; }
    public bool Success { get; set; }

    /// <summary>Código de error de negocio devuelto por Facephi cuando Success = false.</summary>
    public string? ErrorCode { get; set; }
}

// ----- GET /facephi/enrollment/{idT24} -----

/// <summary>
/// Proyección segura del enrolamiento: NO incluye token1, token2 ni extraData.
/// </summary>
public class EnrollmentStatusResponse
{
    public string IdT24 { get; set; } = string.Empty;
    public DocumentType DocumentType { get; set; }
    public int Method { get; set; }
    public EnrollmentStatus Status { get; set; }
    public bool HasFrontDocument { get; set; }
    public bool HasBackDocument { get; set; }
    public string? OperationId { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime UpdatedAt { get; set; }
}

```

7.7 Exceptions/

```

namespace Onboarding.Micro.Person.Exceptions;

/// <summary>No existe un enrolamiento para el idT24 indicado. → HTTP 404</summary>
public class EnrollmentNotFoundException : Exception
{
    public string IdT24 { get; }

    public EnrollmentNotFoundException(string idT24)
        : base($"No existe un enrolamiento para el idT24 indicado.")
        => IdT24 = idT24;
}

/// <summary>La operación no es válida para el estado actual del enrolamiento. → HTTP 409</summary>
public class InvalidEnrollmentStateException : Exception
{
    public InvalidEnrollmentStateException(string message) : base(message) { }
}

/// <summary>Fallo comunicándose con Facephi (timeout, error HTTP,
/// respuesta inesperada). → HTTP 502</summary>
public class FacephiIntegrationException : Exception
{
    public FacephiIntegrationException(string message, Exception? inner = null) : base(message, inner) { }
}

```

7.8 Services/IMongoConnectionService.cs

```
using MongoDB.Driver;

namespace Onboarding.Micro.Person.Services;

/// <summary>
/// Responsable de establecer la conexión con MongoDB y obtener la colección correspondiente.
/// </summary>
public interface IMongoConnectionService
{
    IMongoCollection<T> GetCollection<T>(string collectionName);

    /// <summary>Ping a la base de datos, para health checks.</summary>
    Task<bool> IsHealthyAsync(CancellationTokens cancellationTokens = default);
}
```

7.9 Services/MongoConnectionService.cs

```
using Microsoft.Extensions.Options;
using MongoDB.Bson;
using MongoDB.Driver;
using Onboarding.Micro.Person.Configuration;

namespace Onboarding.Micro.Person.Services;

/// <summary>
/// Conexión a la base de datos MongoDB NUEVA del enrolamiento biométrico.
/// Se registra como SINGLETON: MongoClient mantiene su propio pool de conexiones
/// y es thread-safe; crear uno por request agota los sockets.
/// </summary>
public class MongoConnectionService : IMongoConnectionService
{
    private readonly IMongoDatabase _database;
    private readonly ILogger<MongoConnectionService> _logger;

    public MongoConnectionService(
        IOptions<FacephiMongoSettings> options,
        ILogger<MongoConnectionService> logger)
    {
        _logger = logger;
        var settings = options.Value;

        if (string.IsNullOrEmpty(settings.ConnectionString))
            throw new InvalidOperationException(
                $"{FacephiMongoSettings.SectionName}:ConnectionString no está configurado.");

        if (string.IsNullOrEmpty(settings.DatabaseName))
            throw new InvalidOperationException(
                $"{FacephiMongoSettings.SectionName}:DatabaseName no está configurado.");

        var client = new MongoClient(settings.ConnectionString);
        _database = client.GetDatabase(settings.DatabaseName);

        // Se loguea el nombre de la BD, NUNCA la cadena de conexión (contiene credenciales).
        _logger.LogInformation(
            "Conexión a MongoDB inicializada. BD: {DatabaseName}", settings.DatabaseName);
    }

    public IMongoCollection<T> GetCollection<T>(string collectionName)
        => _database.GetCollection<T>(collectionName);

    public async Task<bool> IsHealthyAsync(CancellationTokens cancellationTokens = default)
    {
        try
        {
            await _database.RunCommandAsync<BsonDocument>(
                new BsonDocument("ping", 1), cancellationTokens: cancellationTokens);
            return true;
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Health check de MongoDB falló.");
            return false;
        }
    }
}
```

```
}  
}
```

7.10 Repositories/IFacephiRepository.cs

```
using Onboarding.Micro.Person.Models;  
  
namespace Onboarding.Micro.Person.Repositories;  
  
/// <summary>  
/// Responsable de almacenar y consultar la información del enrolamiento.  
/// </summary>  
public interface IFacephiRepository  
{  
    Task<FacephiEnrollment> CreateAsync(  
        FacephiEnrollment enrollment, CancellationToken cancellationToken = default);  
  
    Task<FacephiEnrollment?> GetByIdT24Async(  
        string idT24, CancellationToken cancellationToken = default);  
  
    Task<FacephiEnrollment?> UpdateFrontDocumentAsync(  
        string idT24, string token, CancellationToken cancellationToken = default);  
  
    Task<FacephiEnrollment?> UpdateBackDocumentAsync(  
        string idT24, string token, CancellationToken cancellationToken = default);  
  
    Task<FacephiEnrollment?> UpdateTrackingAsync(  
        string idT24, TrackingInfo tracking, EnrollmentStatus status,  
        CancellationToken cancellationToken = default);  
  
    /// <summary>Marca el enrolamiento como fallido sin escribir tracking.</summary>  
    Task<FacephiEnrollment?> UpdateStatusAsync(  
        string idT24, EnrollmentStatus status, CancellationToken cancellationToken = default);  
  
    /// <summary>Crea los índices de la colección. Se invoca una vez al arrancar.</summary>  
    Task EnsureIndexesAsync(CancellationToken cancellationToken = default);  
}
```

7.11 Repositories/FacephiRepository.cs

```
using Microsoft.Extensions.Options;
using MongoDB.Driver;
using Onboarding.Micro.Person.Configuration;
using Onboarding.Micro.Person.Models;
using Onboarding.Micro.Person.Services;

namespace Onboarding.Micro.Person.Repositories;

public class FacephiRepository : IFacephiRepository
{
    private readonly IMongoCollection<FacephiEnrollment> _collection;
    private readonly FacephiMongoSettings _settings;
    private readonly ILogger<FacephiRepository> _logger;

    public FacephiRepository(
        IMongoConnectionService connectionService,
        IOptions<FacephiMongoSettings> options,
        ILogger<FacephiRepository> logger)
    {
        _settings = options.Value;
        _logger = logger;
        _collection = connectionService
            .GetCollection<FacephiEnrollment>(_settings.EnrollmentsCollectionName);
    }

    private static FilterDefinition<FacephiEnrollment> ByIdT24(string idT24)
        => Builders<FacephiEnrollment>.Filter.Eq(e => e.IdT24, idT24);

    private static readonly FindOneAndUpdateOptions<FacephiEnrollment> ReturnUpdated =
        new() { ReturnDocument = ReturnDocument.After };

    /// <summary>
    /// Crea el registro inicial. Es un upsert por idT24: si existía un enrolamiento
    /// no validado, lo reinicia (ver 4.1, idempotencia). El servicio es quien
    /// verifica que el enrolamiento previo no esté ya VALIDATED.
    /// </summary>
    public async Task<FacephiEnrollment> CreateAsync(
        FacephiEnrollment enrollment, CancellationToken cancellationToken = default)
    {
        var now = DateTime.UtcNow;
        enrollment.CreatedAt = now;
        enrollment.UpdatedAt = now;
        enrollment.Status = EnrollmentStatus.CREATED;
        enrollment.Token1 = null;
        enrollment.Token2 = null;
        enrollment.Tracking = null;

        await _collection.ReplaceOneAsync(
            ByIdT24(enrollment.IdT24),
            enrollment,
            new ReplaceOptions { IsUpsert = true },
            cancellationToken);

        _logger.LogInformation("Enrolamiento creado. idT24={IdT24}", enrollment.IdT24);
        return enrollment;
    }
}
```

```

public async Task<FacephiEnrollment?> GetByIdT24Async(
    string idT24, CancellationTokens cancellationTokens = default)
=> await _collection.Find(ByIdT24(idT24)).FirstOrDefaultAsync(cancellationTokens);

public async Task<FacephiEnrollment?> UpdateFrontDocumentAsync(
    string idT24, string token, CancellationTokens cancellationTokens = default)
{
    var update = Builders<FacephiEnrollment>.Update
        .Set(e => e.Token1, token)
        .Set(e => e.Status, EnrollmentStatus.FRONT_SAVED)
        .Set(e => e.UpdatedAt, DateTime.UtcNow);

    var result = await _collection.FindOneAndUpdateAsync(
        ByIdT24(idT24), update, ReturnUpdated, cancellationTokens);

    // Se loguea el hecho, jamás el token.
    _logger.LogInformation("Documento frontal almacenado. idT24={IdT24}", idT24);
    return result;
}

public async Task<FacephiEnrollment?> UpdateBackDocumentAsync(
    string idT24, string token, CancellationTokens cancellationTokens = default)
{
    var update = Builders<FacephiEnrollment>.Update
        .Set(e => e.Token2, token)
        .Set(e => e.Status, EnrollmentStatus.BACK_SAVED)
        .Set(e => e.UpdatedAt, DateTime.UtcNow);

    var result = await _collection.FindOneAndUpdateAsync(
        ByIdT24(idT24), update, ReturnUpdated, cancellationTokens);

    _logger.LogInformation("Documento trasero almacenado. idT24={IdT24}", idT24);
    return result;
}

public async Task<FacephiEnrollment?> UpdateTrackingAsync(
    string idT24, TrackingInfo tracking, EnrollmentStatus status,
    CancellationTokens cancellationTokens = default)
{
    var update = Builders<FacephiEnrollment>.Update
        .Set(e => e.Tracking, tracking)
        .Set(e => e.Status, status)
        .Set(e => e.UpdatedAt, DateTime.UtcNow);

    var result = await _collection.FindOneAndUpdateAsync(
        ByIdT24(idT24), update, ReturnUpdated, cancellationTokens);

    _logger.LogInformation(
        "Tracking actualizado. idT24={IdT24} status={Status} operationId={OperationId}",
        idT24, status, tracking.OperationId);
    return result;
}

public async Task<FacephiEnrollment?> UpdateStatusAsync(
    string idT24, EnrollmentStatus status, CancellationTokens cancellationTokens = default)
{
    var update = Builders<FacephiEnrollment>.Update

```

```

        .Set(e => e.Status, status)
        .Set(e => e.UpdatedAt, DateTime.UtcNow);

    return await _collection.FindOneAndUpdateAsync(
        ByIdT24(idT24), update, ReturnUpdated, cancellationToken);
}

public async Task EnsureIndexesAsync(CancellationToken cancellationToken = default)
{
    var indexes = new List<CreateIndexModel<FacephiEnrollment>>
    {
        // Único por idT24: un enrolamiento vivo por cliente.
        new(Builders<FacephiEnrollment>.IndexKeys.Ascending(e => e.IdT24),
            new CreateIndexOptions { Unique = true, Name = "ux_idT24" }),

        // Soporte y monitoreo.
        new(Builders<FacephiEnrollment>.IndexKeys
            .Ascending(e => e.Status).Descending(e => e.CreatedAt),
            new CreateIndexOptions { Name = "ix_status_createdAt" })
    };

    // TTL: no retener PII biométrica de flujos abandonados (ver sección 9).
    if (_settings.EnrollmentTtlHours > 0)
    {
        indexes.Add(new CreateIndexModel<FacephiEnrollment>(
            Builders<FacephiEnrollment>.IndexKeys.Ascending(e => e.CreatedAt),
            new CreateIndexOptions
            {
                Name = "ttl_createdAt",
                ExpireAfter = TimeSpan.FromHours(_settings.EnrollmentTtlHours)
            }));
    }

    await _collection.Indexes.CreateManyAsync(indexes, cancellationToken);
    _logger.LogInformation("Índices de {Collection} verificados.", _settings.EnrollmentsCollectionName);
}
}

```

Nota sobre el índice único. Con `CreateAsync` haciendo *upsert* por `idT24`, dos peticiones simultáneas para el mismo cliente pueden provocar una `MongoWriteException` por clave duplicada. Es el comportamiento correcto (gana una, la otra falla); si se quiere degradar a reintento, envuelve el `ReplaceOneAsync` en un `try/catch` de `MongoWriteException` con `ServerErrorCode.DuplicateKey` y vuelve a intentar una vez.

7.12 Clients/IFacephiClient.cs

Antes de crear esto, revisa el punto 1 del checklist (§6). Facephi ya está integrado en el micro para onboarding. Si ese cliente puede ejecutar la validación que necesita Soft Token, **úsalo**: adapta `FacephiService` a su interfaz y descarta `IFacephiClient` / `FacephiClient` de este documento. Esta interfaz es la forma mínima que `FacephiService` espera.


```

namespace Onboarding.Micro.Person.Clients;

/// <summary>Datos que Facephi necesita para la validación biométrica.</summary>
public record FacephiValidationRequest(
    string IdT24,
    string DocumentType,
    string FrontDocumentToken,
    string BackDocumentToken,
    string FacialToken,
    int Method);

/// <summary>Resultado de la validación biométrica.</summary>
/// <param name="Success">true si la biometría coincide.</param>
/// <param name="OperationId">Identificador de la operación en Facephi.</param>
/// <param name="ExtraData">Información de tracking devuelta por Facephi.</param>
public record FacephiValidationResult(
    bool Success,
    string? OperationId,
    string? ExtraData,
    string? ErrorCode = null,
    string? ErrorMessage = null);

public interface IFacephiClient
{
    /// <summary>
    /// Ejecuta la validación biométrica comparando la captura facial con los documentos.
    /// </summary>
    /// <exception cref="Exceptions.FacephiIntegrationException">
    /// Si Facephi no responde, responde con error de transporte o con un cuerpo inesperado.
    /// Una validación que responde correctamente "no coincide" NO es una excepción:
    /// se devuelve como Success = false.
    /// </exception>
    Task<FacephiValidationResult> ValidateAsync(
        FacephiValidationRequest request, CancellationToken cancellationToken = default);
}

```

7.13 Clients/FacephiClient.cs — esqueleto

Solo si no hay cliente reutilizable. Los `TODO` marcan lo que depende del contrato real de Facephi, que **todavía no está confirmado** (ver §12).

```

using System.Net.Http.Json;
using System.Text.Json.Serialization;
using Microsoft.Extensions.Options;
using Onboarding.Micro.Person.Configuration;
using Onboarding.Micro.Person.Exceptions;

namespace Onboarding.Micro.Person.Clients;

public class FacephiClient : IFacephiClient
{
    private readonly HttpClient _httpClient;
    private readonly FacephiSettings _settings;
    private readonly ILogger<FacephiClient> _logger;

    public FacephiClient(
        HttpClient httpClient,
        IOptions<FacephiSettings> options,
        ILogger<FacephiClient> logger)
    {
        _httpClient = httpClient;
        _settings = options.Value;
        _logger = logger;
    }

    public async Task<FacephiValidationResult> ValidateAsync(
        FacephiValidationRequest request, CancellationToken cancellationToken = default)
    {
        // TODO: ajustar al contrato real de Facephi (ruta, nombres de campos, autenticación).
        var payload = new
        {
            documentType = request.DocumentType,
            frontDocument = request.FrontDocumentToken,
            backDocument = request.BackDocumentToken,
            selfie = request.FacialToken,
            method = request.Method
        };

        try
        {
            using var response = await _httpClient.PostAsJsonAsync(
                _settings.ValidatePath, payload, cancellationToken);

            if (!response.IsSuccessStatusCode)
            {
                // No se loguea el cuerpo: puede contener datos biométricos.
                _logger.LogError(
                    "Facephi respondió {StatusCode} en la validación. idT24={IdT24}",
                    (int)response.StatusCode, request.IdT24);

                throw new FacephiIntegrationException(
                    $"Facephi respondió con código {(int)response.StatusCode}.");
            }

            var body = await response.Content
                .ReadFromJsonAsync<FacephiValidationApiResponse>(cancellationToken: cancellationToken);

            if (body is null)
                throw new FacephiIntegrationException(

```

```

        "Facephi devolvió un cuerpo vacío o no interpretable.");

    return new FacephiValidationResult(
        Success: body.Success,
        OperationId: body.OperationId,
        ExtraData: body.ExtraData,
        ErrorCode: body.ErrorCode,
        ErrorMessage: body.ErrorMessage);
}
catch (FacephiIntegrationException)
{
    throw;
}
catch (TaskCanceledException ex) when (!cancellationToken.IsCancellationRequested)
{
    throw new FacephiIntegrationException("Timeout invocando a Facephi.", ex);
}
catch (HttpRequestException ex)
{
    throw new FacephiIntegrationException("Error de red invocando a Facephi.", ex);
}
}

// TODO: ajustar a la respuesta real de Facephi.
private class FacephiValidationApiResponse
{
    [JsonPropertyName("success")] public bool Success { get; set; }
    [JsonPropertyName("operationId")] public string? OperationId { get; set; }
    [JsonPropertyName("extraData")] public string? ExtraData { get; set; }
    [JsonPropertyName("errorCode")] public string? ErrorCode { get; set; }
    [JsonPropertyName("errorMessage")] public string? ErrorMessage { get; set; }
}
}

```

FacephiSettings (reutilizar la del micro si existe):

```

namespace Onboarding.Micro.Person.Configuration;

public class FacephiSettings
{
    public const string SectionName = "Facephi";

    public string BaseUrl { get; set; } = string.Empty;
    public string ValidatePath { get; set; } = "/api/v1/validate"; // TODO: confirmar
    public string ApiKey { get; set; } = string.Empty; // desde vault / env
    public int TimeoutSeconds { get; set; } = 30;
}

```

7.14 Clients/IAlfrescoClient.cs — fase posterior

```
namespace Onboarding.Micro.Person.Clients;

/// <summary>
/// Cliente para el almacenamiento del contrato firmado y las evidencias biométricas
/// en Alfresco. IMPLEMENTACIÓN POSTERIOR – fuera del alcance de APP-5258.
/// Se define ahora para que el punto de extensión exista y no haya refactor después.
/// </summary>
public interface IAlfrescoClient
{
    Task<string> UploadSignedContractAsync(
        string idT24, byte[] content, string fileName, CancellationToken cancellationToken = default);

    Task<string> UploadBiometricEvidenceAsync(
        string idT24, byte[] content, string fileName, CancellationToken cancellationToken = default);
}

/// <summary>Stub. Falla explícitamente para que nadie asuma que ya funciona.</summary>
public class AlfrescoClient : IAlfrescoClient
{
    private const string NotImplementedMessage =
        "La integración con Alfresco es una fase posterior a APP-5258.";

    public Task<string> UploadSignedContractAsync(
        string idT24, byte[] content, string fileName, CancellationToken cancellationToken = default)
        => throw new NotImplementedException(NotImplementedMessage);

    public Task<string> UploadBiometricEvidenceAsync(
        string idT24, byte[] content, string fileName, CancellationToken cancellationToken = default)
        => throw new NotImplementedException(NotImplementedMessage);
}
```

7.15 Services/IFacephiService.cs

```
using Onboarding.Micro.Person.Dtos;

namespace Onboarding.Micro.Person.Services;

/// <summary>
/// Responsable de orquestar el proceso de enrolamiento biométrico.
/// </summary>
public interface IFacephiService
{
    Task<CreateEnrollmentResponse> CreateEnrollmentAsync(
        CreateEnrollmentRequest request, CancellationToken cancellationToken = default);

    Task<SaveDocumentResponse> SaveFrontDocumentAsync(
        SaveDocumentRequest request, CancellationToken cancellationToken = default);

    Task<SaveDocumentResponse> SaveBackDocumentAsync(
        SaveDocumentRequest request, CancellationToken cancellationToken = default);

    Task<ValidateResponse> ValidateFacephiAsync(
        ValidateRequest request, CancellationToken cancellationToken = default);

    Task<EnrollmentStatusResponse> GetEnrollmentStatusAsync(
        string idT24, CancellationToken cancellationToken = default);
}
```

7.16 Services/FacephiService.cs

```
using Microsoft.Extensions.Options;
using Onboarding.Micro.Person.Clients;
using Onboarding.Micro.Person.Configuration;
using Onboarding.Micro.Person.Dtos;
using Onboarding.Micro.Person.Exceptions;
using Onboarding.Micro.Person.Models;
using Onboarding.Micro.Person.Repositories;

namespace Onboarding.Micro.Person.Services;

public class FacephiService : IFacephiService
{
    private readonly IFacephiRepository _repository;
    private readonly IFacephiClient _facephiClient;
    private readonly FacephiMongoSettings _settings;
    private readonly ILogger<FacephiService> _logger;

    public FacephiService(
        IFacephiRepository repository,
        IFacephiClient facephiClient,
        IOptions<FacephiMongoSettings> options,
        ILogger<FacephiService> logger)
    {
        _repository = repository;
        _facephiClient = facephiClient;
        _settings = options.Value;
        _logger = logger;
    }

    // ----- 1. Crear enrolamiento -----

    public async Task<CreateEnrollmentResponse> CreateEnrollmentAsync(
        CreateEnrollmentRequest request, CancellationToken cancellationToken = default)
    {
        var existing = await _repository.GetByIdT24Async(request.IdT24, cancellationToken);

        if (existing is { Status: EnrollmentStatus.VALIDATED or EnrollmentStatus.CONTRACT_STORED })
            throw new InvalidEnrollmentStateException(
                "Ya existe un enrolamiento validado para este cliente.");

        if (existing is not null)
            _logger.LogInformation(
                "Se reinicia un enrolamiento previo no validado. idT24={IdT24} statusPrevio={Status}",
                request.IdT24, existing.Status);

        var enrollment = await _repository.CreateAsync(new FacephiEnrollment
        {
            IdT24 = request.IdT24,
            DocumentType = request.DocumentType,
            Method = request.Method
        }, cancellationToken);

        return new CreateEnrollmentResponse
        {
            IdT24 = enrollment.IdT24,
            Status = enrollment.Status,
        }
    }
}
```

```

        CreatedAt = enrollment.CreatedAt
    };
}

// ----- 2 y 3. Documentos frontal y trasero -----

public async Task<SaveDocumentResponse> SaveFrontDocumentAsync(
    SaveDocumentRequest request, CancellationToken cancellationToken = default)
{
    ValidateTokenSize(request.Token);
    var enrollment = await GetOrThrowAsync(request.IdT24, cancellationToken);
    EnsureNotFinalized(enrollment);

    var updated = await _repository.UpdateFrontDocumentAsync(
        request.IdT24, request.Token, cancellationToken)
        ?? throw new EnrollmentNotFoundException(request.IdT24);

    return new SaveDocumentResponse { IdT24 = updated.IdT24, Status = updated.Status };
}

public async Task<SaveDocumentResponse> SaveBackDocumentAsync(
    SaveDocumentRequest request, CancellationToken cancellationToken = default)
{
    ValidateTokenSize(request.Token);
    var enrollment = await GetOrThrowAsync(request.IdT24, cancellationToken);
    EnsureNotFinalized(enrollment);

    // El dorso solo tiene sentido con el frente ya guardado.
    if (!enrollment.HasFrontDocument)
        throw new InvalidEnrollmentStateException(
            "Debe almacenarse primero la imagen frontal del documento.");

    var updated = await _repository.UpdateBackDocumentAsync(
        request.IdT24, request.Token, cancellationToken)
        ?? throw new EnrollmentNotFoundException(request.IdT24);

    return new SaveDocumentResponse { IdT24 = updated.IdT24, Status = updated.Status };
}

// ----- 4. Validación biométrica -----

public async Task<ValidateResponse> ValidateFacephiAsync(
    ValidateRequest request, CancellationToken cancellationToken = default)
{
    ValidateTokenSize(request.FacialToken);
    var enrollment = await GetOrThrowAsync(request.IdT24, cancellationToken);

    if (enrollment.Status is EnrollmentStatus.VALIDATED or EnrollmentStatus.CONTRACT_STORED)
        throw new InvalidEnrollmentStateException("El enrolamiento ya fue validado.");

    if (!enrollment.HasFrontDocument || !enrollment.HasBackDocument)
        throw new InvalidEnrollmentStateException(
            "Deben almacenarse ambas imágenes del documento antes de validar.");

    var result = await _facephiClient.ValidateAsync(new FacephiValidationRequest(
        IdT24: enrollment.IdT24,
        DocumentType: enrollment.DocumentType.ToString(),
        FrontDocumentToken: enrollment.Token1!,

```

```

        BackDocumentToken: enrollment.Token2!,
        FacialToken: request.FacialToken,
        Method: enrollment.Method), cancellationToken);

if (!result.Success)
{
    await _repository.UpdateStatusAsync(
        request.IdT24, EnrollmentStatus.VALIDATION_FAILED, cancellationToken);

    _logger.LogWarning(
        "Validación biométrica no exitosa. idT24={IdT24} errorCode={ErrorCode}",
        request.IdT24, result.ErrorCode);

    return new ValidateResponse
    {
        IdT24 = request.IdT24,
        Status = EnrollmentStatus.VALIDATION_FAILED,
        Success = false,
        ErrorCode = result.ErrorCode
    };
}

// 5. Validación exitosa → se actualiza el tracking y el flujo continúa
// con el enrolamiento del Soft Token (APP-5223).
var tracking = new TrackingInfo
{
    OperationId = result.OperationId,
    ExtraData = result.ExtraData
};

var updated = await _repository.UpdateTrackingAsync(
    request.IdT24, tracking, EnrollmentStatus.VALIDATED, cancellationToken)
    ?? throw new EnrollmentNotFoundException(request.IdT24);

return new ValidateResponse
{
    IdT24 = updated.IdT24,
    Status = updated.Status,
    OperationId = updated.Tracking?.OperationId,
    Success = true
};
}

// ----- 5. Consulta de estado -----

public async Task<EnrollmentStatusResponse> GetEnrollmentStatusAsync(
    string idT24, Cancellation token cancellationToken = default)
{
    var enrollment = await GetOrThrowAsync(idT24, cancellationToken);

    // Proyección segura: sin token1, token2 ni extraData.
    return new EnrollmentStatusResponse
    {
        IdT24 = enrollment.IdT24,
        DocumentType = enrollment.DocumentType,
        Method = enrollment.Method,
        Status = enrollment.Status,
        HasFrontDocument = enrollment.HasFrontDocument,
    };
}

```



```

        HasBackDocument = enrollment.HasBackDocument,
        OperationId = enrollment.Tracking?.OperationId,
        CreatedAt = enrollment.CreatedAt,
        UpdatedAt = enrollment.UpdatedAt
    };
}

// ----- Auxiliares -----

private async Task<FacephiEnrollment> GetOrThrowAsync(
    string idT24, CancellationToken cancellationToken)
    => await _repository.GetByIdT24Async(idT24, cancellationToken)
        ?? throw new EnrollmentNotFoundException(idT24);

private static void EnsureNotFinalized(FacephiEnrollment enrollment)
{
    if (enrollment.Status is EnrollmentStatus.VALIDATED or EnrollmentStatus.CONTRACT_STORED)
        throw new InvalidEnrollmentStateException(
            "El enrolamiento ya fue validado; no admite más documentos.");
}

private void ValidateTokenSize(string token)
{
    if (token.Length > _settings.MaxTokenLengthBytes)
        throw new ArgumentException(
            $"El token excede el tamaño máximo permitido ({_settings.MaxTokenLengthBytes} bytes).");
}
}

```

Variante sin el campo `status` (si el equipo rechaza la recomendación de §3.1): sustituye los accesos a `enrollment.Status` por un método privado que infiera el estado a partir de `Tracking`, `Token2` y `Token1`, en ese orden, y elimina los `Set(e => e.Status, ...)` del repositorio. El resto del servicio no cambia.

7.17 Controllers/FacephiController.cs

```
using Microsoft.AspNetCore.Mvc;
using Onboarding.Micro.Person.Dtos;
using Onboarding.Micro.Person.Services;

namespace Onboarding.Micro.Person.Controllers;

[ApiController]
[Route("facephi")]
[Produces("application/json")]
// TODO (§12): definir con el equipo si estos endpoints llevan [Authorize] con la misma
// política del resto del micro, o si quedan internos detrás del gateway/BFF.
// [Authorize]
public class FacephiController : ControllerBase
{
    private readonly IFacephiService _facephiService;

    public FacephiController(IFacephiService facephiService) => _facephiService = facephiService;

    /// <summary>Crea el registro inicial del enrolamiento.</summary>
    [HttpPost("enrollment")]
    [ProducesResponseType(typeof(CreateEnrollmentResponse), StatusCodes.Status201Created)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status409Conflict)]
    public async Task<IActionResult> CreateEnrollment(
        [FromBody] CreateEnrollmentRequest request, CancellationToken cancellationToken)
    {
        var response = await _facephiService.CreateEnrollmentAsync(request, cancellationToken);

        return CreatedAtAction(
            nameof(GetEnrollment),
            new { idT24 = response.IdT24 },
            response);
    }

    /// <summary>Guarda el token de la imagen frontal del documento.</summary>
    [HttpPost("document/front")]
    [ProducesResponseType(typeof(SaveDocumentResponse), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    [ProducesResponseType(StatusCodes.Status409Conflict)]
    public async Task<IActionResult> SaveFrontDocument(
        [FromBody] SaveDocumentRequest request, CancellationToken cancellationToken)
    => Ok(await _facephiService.SaveFrontDocumentAsync(request, cancellationToken));

    /// <summary>Guarda el token de la imagen trasera del documento.</summary>
    [HttpPost("document/back")]
    [ProducesResponseType(typeof(SaveDocumentResponse), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    [ProducesResponseType(StatusCodes.Status409Conflict)]
    public async Task<IActionResult> SaveBackDocument(
        [FromBody] SaveDocumentRequest request, CancellationToken cancellationToken)
    => Ok(await _facephiService.SaveBackDocumentAsync(request, cancellationToken));

    /// <summary>
    /// Ejecuta la validación biométrica utilizando los documentos almacenados y la captura facial.

```

```

/// </summary>
[HttpPost("validate")]
[ProducesResponseType(typeof(ValidateResponse), StatusCodes.Status200OK)]
[ProducesResponseType(typeof(ValidateResponse), StatusCodes.Status422UnprocessableEntity)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status409Conflict)]
[ProducesResponseType(StatusCodes.Status502BadGateway)]
public async Task<IActionResult> Validate(
    [FromBody] ValidateRequest request, CancellationToken cancellationToken)
{
    var response = await _facephiService.ValidateFacephiAsync(request, cancellationToken);

    // La biometría no coincide: no es un error del servicio, es un resultado de negocio.
    return response.Success ? Ok(response) : UnprocessableEntity(response);
}

/// <summary>Consulta el estado del proceso.</summary>
[HttpGet("enrollment/{idT24}")]
[ProducesResponseType(typeof(EnrollmentStatusResponse), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> GetEnrollment(
    string idT24, CancellationToken cancellationToken)
    => Ok(await _facephiService.GetEnrollmentStatusAsync(idT24, cancellationToken));
}

```

7.18 Services/MongoIndexInitializer.cs

```
using Onboarding.Micro.Person.Repositories;

namespace Onboarding.Micro.Person.Services;

/// <summary>
/// Crea los índices de la colección al arrancar. No tumba el servicio si falla:
/// un micro que no arranca por un índice es peor que un índice ausente.
/// </summary>
public class MongoIndexInitializer : IHostedService
{
    private readonly IServiceProvider _serviceProvider;
    private readonly ILogger<MongoIndexInitializer> _logger;

    public MongoIndexInitializer(
        IServiceProvider serviceProvider, ILogger<MongoIndexInitializer> logger)
    {
        _serviceProvider = serviceProvider;
        _logger = logger;
    }

    public async Task StartAsync(CancellationTokens cancellationTokens)
    {
        try
        {
            using var scope = _serviceProvider.CreateScope();
            var repository = scope.ServiceProvider.GetRequiredService<IFacephiRepository>();
            await repository.EnsureIndexesAsync(cancellationTokens);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "No se pudieron crear los índices de facephi_enrollments.");
        }
    }

    public Task StopAsync(CancellationTokens cancellationTokens) => Task.CompletedTask;
}
```

7.19 Middleware/ExceptionHandlerMiddleware.cs

Si el micro ya tiene un middleware de excepciones (punto 4 del checklist), no añadas este: registra ahí el mapeo de las tres excepciones nuevas.

```

using System.Text.Json;
using Microsoft.AspNetCore.Mvc;
using Onboarding.Micro.Person.Exceptions;

namespace Onboarding.Micro.Person.Middleware;

public class ExceptionHandlingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionHandlingMiddleware> _logger;

    public ExceptionHandlingMiddleware(
        RequestDelegate next, ILogger<ExceptionHandlingMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (Exception ex)
        {
            await HandleAsync(context, ex);
        }
    }

    private async Task HandleAsync(HttpContext context, Exception exception)
    {
        var (status, title) = exception switch
        {
            EnrollmentNotFoundException =>
                (StatusCodes.Status404NotFound, "Enrolamiento no encontrado"),
            InvalidEnrollmentStateException =>
                (StatusCodes.Status409Conflict, "Estado inválido para la operación"),
            FacephiIntegrationException =>
                (StatusCodes.Status502BadGateway, "Error de integración con Facephi"),
            ArgumentException =>
                (StatusCodes.Status400BadRequest, "Solicitud inválida"),
            _ =>
                (StatusCodes.Status500InternalServerError, "Error interno")
        };

        if (status >= 500)
            _logger.LogError(exception, "Error no controlado en {Path}", context.Request.Path);
        else
            _logger.LogWarning("{Title} en {Path}: {Message}",
                title, context.Request.Path, exception.Message);

        var problem = new ProblemDetails
        {
            Status = status,
            Title = title,
            // En 5xx no se filtra el detalle interno al cliente.
            Detail = status >= 500 ? "Ocurrió un error procesando la solicitud." : exception.Message,
        };
    }
}

```

```
        Instance = context.Request.Path
    };

    context.Response.StatusCode = status;
    context.Response.ContentType = "application/problem+json";
    await context.Response.WriteAsync(JsonSerializer.Serialize(problem));
}
}
```

8. Configuración e integración

8.1 Líneas a añadir en `Program.cs`

No reemplaces el `Program.cs` del micro. Añade únicamente estos bloques.

```
using Onboarding.Micro.Person.Clients;
using Onboarding.Micro.Person.Configuration;
using Onboarding.Micro.Person.Middleware;
using Onboarding.Micro.Person.Repositories;
using Onboarding.Micro.Person.Services;

// — Configuración —————
builder.Services.Configure<FacephiMongoSettings>(
    builder.Configuration.GetSection(FacephiMongoSettings.SectionName));

builder.Services.Configure<FacephiSettings>(
    builder.Configuration.GetSection(FacephiSettings.SectionName));

// — Mongo (BD nueva) —————
// Singleton: MongoClient es thread-safe y gestiona su propio pool.
builder.Services.AddSingleton<IMongoConnectionService, MongoConnectionService>();

// — Persistencia y negocio —————
builder.Services.AddScoped<IFacephiRepository, FacephiRepository>();
builder.Services.AddScoped<IFacephiService, FacephiService>();

// — Clientes externos —————
// ▲ Si el micro YA tiene un cliente Facephi registrado, NO añadas este bloque:
//   reutiliza el existente y adapta FacephiService a su interfaz.
builder.Services.AddHttpClient<IFacephiClient, FacephiClient>((sp, client) =>
{
    var settings = sp.GetRequiredService<IOptions<FacephiSettings>>().Value;
    client.BaseAddress = new Uri(settings.BaseUrl);
    client.Timeout = TimeSpan.FromSeconds(settings.TimeoutSeconds);
    client.DefaultRequestHeaders.Add("x-api-key", settings.ApiKey); // TODO: confirmar esquema de auth
});

builder.Services.AddSingleton<IAlfrescoClient, AlfrescoClient>(); // stub, fase posterior

// — Índices al arrancar —————
builder.Services.AddHostedService<MongoIndexInitializer>();

// — Pipeline (solo si el micro no tiene ya un middleware de excepciones) ———
app.UseMiddleware<ExceptionHandlingMiddleware>();
```

8.2 Sección a añadir en `appsettings.json`

```
{
  "FacephiMongo": {
    "ConnectionString": "",
    "DatabaseName": "facephi_softtoken",
    "EnrollmentsCollectionName": "facephi_enrollments",
    "EnrollmentTtlHours": 24,
    "MaxTokenLengthBytes": 8388608
  },
  "Facephi": {
    "BaseUrl": "",
    "ValidatePath": "/api/v1/validate",
    "ApiKey": "",
    "TimeoutSeconds": 30
  }
}
```

`ConnectionString`, `BaseUrl` y `ApiKey` quedan **vacíos en el repositorio**. Se inyectan por variable de entorno o desde el vault del banco: `FacephiMongo__ConnectionString`, `Facephi__BaseUrl`, `Facephi__ApiKey`.

9. Seguridad y datos biométricos

`token1` , `token2` , el token facial y `extraData` son **datos biométricos**: la categoría más sensible de dato personal. Reglas no negociables:

Regla	Cómo se cumple en este diseño
Nunca loguear tokens	Todos los <code>ILogger</code> de §7 registran solo <code>idT24</code> , <code>status</code> , <code>operationId</code> y duración. El cuerpo de las respuestas de Facephi tampoco se loguea
Nunca exponerlos por API	<code>EnrollmentStatusResponse</code> es una proyección con banderas booleanas; los tokens no salen del micro
Credenciales fuera del repo	<code>ConnectionString</code> y <code>ApiKey</code> vacíos en <code>appsettings.json</code> , inyectados por entorno/vault
No retener PII innecesaria	Índice TTL sobre <code>createdAt</code> (24 h configurable)
Límite de tamaño	<code>MaxTokenLengthBytes</code> validado en el servicio → 400 si se excede
Transporte cifrado	HTTPS obligatorio hacia Facephi; TLS en la conexión a Mongo

Límite de 16 MB por documento en MongoDB. `token1` + `token2` son cadenas base64 de imágenes. Dos imágenes grandes pueden acercarse peligrosamente al límite y el fallo aparecería en producción, no en pruebas. Por eso el `MaxTokenLengthBytes` está en 8 MB por defecto: ajústalo al tamaño real que devuelve el SDK de Facephi tras medirlo. Si los tokens resultan ser más grandes, la alternativa es GridFS o almacenamiento externo referenciado.

Cifrado en reposo. Queda como pregunta para el equipo de seguridad (§12): si la política del banco exige cifrado a nivel de campo, MongoDB CSFLE (*Client-Side Field Level Encryption*) sobre `token1` / `token2` es la opción estándar y no cambia el resto del diseño.

Trazabilidad. Cada operación deja un log con `idT24` y `operationId` , suficiente para auditar el flujo completo sin exponer ni un byte de biometría.

10. Pruebas y verificación

10.1 Local

```
# 1. Mongo local
docker run -d --name mongo-facephi -p 27017:27017 mongo:7

# 2. Variables de entorno
export FacephiMongo__ConnectionString="mongodb://localhost:27017"
export FacephiMongo__DatabaseName="facephi_softtoken"

# 3. Compilar y ejecutar
dotnet build
dotnet run

# 4. Swagger
# https://localhost:5001/swagger
```

10.2 Flujo feliz con curl

```
BASE=https://localhost:5001

# 1. Crear enrolamiento
curl -sX POST $BASE/facephi/enrollment \
  -H "Content-Type: application/json" \
  -d '{"idT24":"123456789","documentType":"CED","method":1}'
# → 201 { "idT24":"123456789", "status":"CREATED", ... }

# 2. Documento frontal
curl -sX POST $BASE/facephi/document/front \
  -H "Content-Type: application/json" \
  -d '{"idT24":"123456789","token":"TOKEN_FRONTAL_DE_PRUEBA"}'
# → 200 { "status":"FRONT_SAVED" }

# 3. Documento trasero
curl -sX POST $BASE/facephi/document/back \
  -H "Content-Type: application/json" \
  -d '{"idT24":"123456789","token":"TOKEN TRASERO_DE_PRUEBA"}'
# → 200 { "status":"BACK_SAVED" }

# 4. Estado
curl -s $BASE/facephi/enrollment/123456789
# → 200 { "status":"BACK_SAVED", "hasFrontDocument":true, "hasBackDocument":true }
# Verificar que NO aparecen token1 ni token2 en la respuesta.

# 5. Validación (requiere el cliente Facephi real)
curl -sX POST $BASE/facephi/validate \
  -H "Content-Type: application/json" \
  -d '{"idT24":"123456789","facialToken":"TOKEN_FACIAL_DE_PRUEBA"}'
```

10.3 Casos negativos obligatorios

Caso	Resultado esperado
Guardar el dorso sin haber guardado el frente	409
GET de un idT24 inexistente	404
POST /validate sin ambos documentos	409
POST /enrollment con documentType inválido	400
POST /enrollment repetido con enrolamiento ya VALIDATED	409
POST /enrollment repetido con enrolamiento a medias	201 , registro reiniciado
Token que excede MaxTokenLengthBytes	400
Facephi caído durante /validate	502
Facephi responde "no coincide"	422 , estado VALIDATION_FAILED

10.4 Verificación en Mongo

```
use facephi_softtoken
db.facephi_enrollments.findOne({ idT24: "123456789" })
db.facephi_enrollments.getIndexes() // ux_idT24, ttl_createdAt, ix_status_createdAt
```

El documento debe coincidir con el modelo de §3.

10.5 Revisión de logs

```
grep -i "TOKEN_FRONTAL_DE_PRUEBA\|TOKEN_FACIAL_DE_PRUEBA" logs/*
# No debe devolver ninguna línea. Si devuelve algo, hay una fuga de PII biométrica.
```

10.6 Pruebas unitarias sugeridas (xUnit + Moq)

```
[Fact]
public async Task SaveBackDocumentAsync_SinDocumentoFrontal_LanzaInvalidEnrollmentState()
{
    var repository = new Mock<IFacephiRepository>();
    repository.Setup(r => r.GetById24Async("123", It.IsAny<CancellationToken>()))
        .ReturnsAsync(new FacephiEnrollment
        {
            IdT24 = "123",
            Status = EnrollmentStatus.CREATED,
            Token1 = null
        });

    var service = new FacephiService(
        repository.Object,
        Mock.Of<IFacephiClient>(),
        Options.Create(new FacephiMongoSettings { MaxTokenLengthBytes = 1024 }),
        NullLogger<FacephiService>.Instance);

    await Assert.ThrowsAsync<InvalidEnrollmentStateException>(() =>
        service.SaveBackDocumentAsync(new SaveDocumentRequest { IdT24 = "123", Token = "abc" }));
}
```

Casos mínimos a cubrir: cada transición inválida de la tabla de §5.1, validación exitosa (persiste tracking y deja `VALIDATED`), validación fallida (deja `VALIDATION_FAILED` y no escribe tracking), y `FacephiIntegrationException` propagada como `502`. Para integración, `Testcontainers.MongoDb` levanta un Mongo real por prueba.

11. Subtareas y estimación

#	Subtarea	Alcance	Est.
1	Descubrimiento e integración	Checklist §6, decidir qué se reutiliza, aprovisionar la BD nueva	1 pt
2	Modelos y persistencia	§7.1-7.11: modelos, <code>MongoConnectionService</code> , <code>FacephiRepository</code> , índices	3 pts
3	Endpoints y contratos	§7.6, §7.17, §7.19: DTOs, controlador, manejo de errores	3 pts
4	Orquestación	§7.15-7.16: <code>FacephiService</code> , máquina de estados	2 pts
5	Integración Facephi	Conectar/extender el cliente existente para la validación	5 pts
6	Pruebas	Unitarias de servicio + integración con <code>Testcontainers</code>	3 pts
7	Integración Alfresco	Contrato firmado y evidencias — fase posterior	5 pts

Subtareas 1-6 = **17 puntos** para cerrar el alcance de APP-5258. La 5 depende de que Facephi entregue el contrato de la operación de validación; si se retrasa, las 1-4 y 6 avanzan contra el *stub* sin bloquearse.

Dependencias externas

- Aprovisionamiento de la base de datos MongoDB nueva (infraestructura).
- Contrato y credenciales de la operación de validación de Facephi.
- Decisión de seguridad sobre autenticación de los endpoints.
- APP-5223 consume la salida de este trabajo: acordar con esa historia cómo se le notifica que el enrolamiento quedó `VALIDATED` .

12. Preguntas abiertas

Cerrarlas antes o durante la implementación. Las cuatro primeras son **bloqueantes** para terminar.

#	Pregunta	Para quién	Bloquea
1	¿Los endpoints <code>/facephi/*</code> llevan <code>[Authorize]</code> con la política del micro, o quedan internos tras el gateway? ¿El <code>idT24</code> sale del JWT o del body?	Arquitectura / Seguridad	Sí
2	¿Cuál es el contrato exacto de Facephi para la validación: URL, esquema de autenticación, nombres de campos y semántica de <code>extraData</code> ?	Facephi / Integraciones	Sí
3	¿El cliente Facephi que ya usa onboarding expone la operación que necesita Soft Token, o hay que añadirle un método? ¿Mismas credenciales y <code>tenant</code> ?	Equipo del micro	Sí
4	Nombre, credenciales y responsable del aprovisionamiento de la BD Mongo nueva	Infraestructura	Sí
5	¿Se aprueban <code>status</code> , <code>createdAt</code> y <code>updatedAt</code> en el documento (recomendación §3.1)?	Equipo / Arquitectura	No
6	¿ <code>method</code> es un catálogo cerrado? ¿Qué valores además de <code>1</code> ?	Producto / Facephi	No
7	¿ <code>/document/front</code> y <code>/document/back</code> se mantienen separados o se unifican? (el ticket los marca <i>tentativos</i>)	Producto / Frontend	No
8	Política de retención: ¿el registro debe sobrevivir a la activación del Soft Token? ¿Qué TTL? ¿Se limpian <code>token1</code> / <code>token2</code> tras validar?	Seguridad / Cumplimiento	No
9	¿Se exige cifrado en reposo a nivel de campo (CSFLE) para los tokens biométricos?	Seguridad	No
10	¿Cómo se notifica a APP-5223 que el enrolamiento quedó validado: consulta al <code>GET</code> , evento, o llamada directa?	Equipo APP-5223	No

Documento generado como planeación técnica de APP-5258. El código incluido no ha sido compilado: requiere ajuste de namespaces y validación contra el repositorio real de `onboarding-micro-person` .